

Smart MCQ Solver

Varun Agnihotri | Roll No: 24f2004142 | BSDA2001P Deep Learning & GenAI Project

GitHub: [PythonicVarun/GenAI_Project-May_2026](https://github.com/PythonicVarun/GenAI_Project-May_2026) | W&B: [varunagnihotri/24f2004142-t22026](https://weightsandbiases.com/run/varunagnihotri/24f2004142-t22026) | Kaggle: [pythonicvarun](https://kaggle.com/pythonicvarun)

1. Abstract

The Smart MCQ Solver Challenge asks for the top three answer choices for scientific and technical multiple-choice questions, scored by MAP@3. I built three models: Gemma-4 E4B IT fine-tuned with LoRA, a BiLSTM scorer written from scratch in PyTorch, and a TF-IDF + logistic regression baseline. All three are fed by a TF-IDF retriever that pulls similar solved questions from the training set and prepends them to the prompt.

My best public score is **0.75644** (BiLSTM), with Gemma-4 at 0.74189 and the baseline at 0.47173, placing me **381st of 1,531 teams**. The two deep models reported near-perfect validation (0.995 and 0.998) and then lost about 0.25 on the leaderboard.

The main contribution of this report is the diagnosis of that gap. The 2,000 training rows are template paraphrases of only **415 distinct questions**, so a random row-level split leaves a paraphrase of nearly every validation row in the training half, and the retriever hands it to the model with the answer attached. The fine-tuned model reproduces a train-derived lookup table on 99.6% of the rows that table determines, and all 1,180 teams above the cutoff finished inside a band 0.047 wide: the task has a ceiling near 0.78 that both deep models had reached.

2. Introduction

Problem statement: Each row is a prompt, five options A to E, and, in the training file, the correct letter. Training is 2,000 rows, test is 500. A submission gives three letters per row, best first, scored by MAP@3: 1 if the first guess is right, 1/2 if the second, 1/3 if the third, else 0.

Objective: Clear the 0.73 cutoff and build three genuinely different models rather than three variants of one idea: one pretrained LLM fine-tuned, one deep model from scratch, one classical baseline, with all experiments tracked in Weights & Biases and the pipeline reproducible from the repository.

3. Dataset & Preprocessing

Description: 2,000 training and 500 test rows, columns id, prompt, A..E, answer, no missing values. Options are paragraph-length, topics span philosophy, plasma physics, cartography and chemistry, and answer letters are mildly imbalanced (B 24.5%, C 23.0%, A 18.4%, D 17.9%, E 16.2%).

EDA: the corpus is smaller than it looks: Prompts wrap a core question in an instruction template, e.g. *"Choose the correct answer: What is the definition of an improper rotation?"* and *"Select the most accurate option: ...that same question... carefully."* Six prefixes and five suffixes account for all 2,000 rows and none negates the question. Stripping them leaves **415 distinct core questions**, in groups of 1 to 18 rows. Two properties follow, measured over all 2,000 rows:

1. The answer letter is constant within every core group (415 of 415), so paraphrasing never reshuffles options.
2. Of 588 distinct ordered option sequences, all 588 map to exactly one answer letter.

Property 2 means a dictionary {option sequence -> answer} built from the training file solves any row whose option sequence has been seen. Section 6.3 shows this is what my models converged to.

Distractor difficulty: Taking the highest pairwise character similarity among the five options per row (difflib.SequenceMatcher, 600 train / 400 test rows), the mean is 0.82 on train and 0.81 on test, and about a third of rows contain two options over 95% identical. One item's five options differ only in

light/heavy ion, easy/difficult, 10/100 kV and fusion/fission: well-posed, but needing three fine-grained differences resolved across near-identical paragraphs. This is why prompt-to-option similarity carries so little signal.

Preprocessing: Two shared functions in `src/local/utils.py`: `clean()` lowercases, strips non-alphanumerics and collapses whitespace; `extract_core_question()` splits on the first colon and truncates at the last question mark, giving the retrieval key. The self-RAG step queries a TF-IDF index over the training set and prepends the top 3 matches as Q: <question> / Answer: <letter>) <text>, excluding the row's own index during training. **No augmentation was applied:** the paraphrase templates already are augmentation.

4. Tokenization Strategy

Primary (best-scoring path): word-level vocabulary built from scratch. `src/local/vocab.py` counts word frequencies over prompts and all five options, reserves <PAD>, <UNK>, <SEP>, and caps at 20,000 entries. The cleaned corpus yields only **2,976 distinct words**, so the cap never binds.

Justification. The vocabulary is small and closed, so subword tokenization gains little over whole words, and a 2,976 x 128 embedding table (381k parameters) is trainable from scratch on 1,700 rows where a 30k BPE vocabulary would have meant 3.8M.

Implementation: the sequence budget. Each (question, context, option) triple must fit `MAX_SEQ_LEN = 256`. Truncating from the right would consume the option, which is the deciding part, so `build_triple()` encodes question and option at full length first and gives the remaining slack to the retrieved context, falling back to a 40/60 question/option split if the two alone overflow. Final layout is question <SEP> context <SEP> option.

Secondary: Gemma-4's SentencePiece tokenizer, used unchanged. The MCQ is wrapped in Gemma's chat template with the assistant turn being the single answer letter. At inference I do not generate: one forward pass, then rank the log-probabilities of the five single tokens for A to E at the last position. This gives the top-3 directly and is five times cheaper than scoring each option separately.

Experimental comparison: The budgeting mattered more than the length. Rewriting `build_triple` to protect the question and option (6bb2185) moved validation `MAP@3` from 0.9522 to 1.0000 and the leaderboard from 0.72527 to 0.75436, all at length 128; doubling to 256 afterwards changed neither. For Gemma I capped sequences at 512 to fit the Kaggle GPU, a known limitation: that long prompts lose the answer end since, TRL truncates from the right, where the answer token sits.

5. Modeling & Experimentation

5.1 Model 1: Custom Deep Learning Model / RNN-Based Model (BiLSTM scorer)

Model 1: BiLSTM scorer, built from scratch in PyTorch (2.0 M parameters, 2,976-word vocabulary)

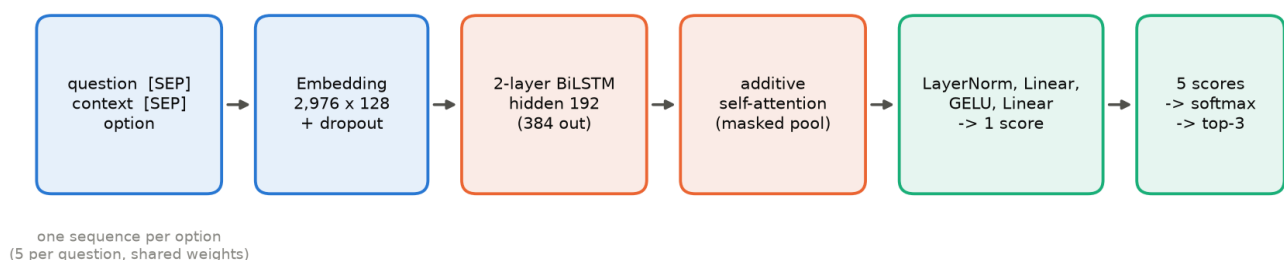


Figure 1: BiLSTM architecture

Architecture: (src/local/model.py). Each question becomes five sequences, one per option, scored by shared weights: (1) embedding 2,976 x 128, padding_idx=0, dropout 0.35; (2) two-layer bidirectional LSTM, hidden 192 per direction, packed so padding never enters the recurrence; (3) additive self-attention over timesteps, $v(\tanh(W.h))$, masked at pad positions, softmax-weighted into one 384-vector; (4) head of LayerNorm, Linear(384->192), GELU, Dropout, Linear(192->1), giving one scalar per option; (5) the five scalars form a 5-way logit vector trained with cross-entropy, softmaxed and ranked at inference. **Total 1,986,433 parameters**, 381k of it the embedding table.

Salient points: The 5-way softmax over shared-weight option scores is the standard multiple-choice head (the shape BertForMultipleChoice uses); it forces the model to compare options against each other rather than judge each in isolation, which is what MAP@3 rewards. Bidirectionality matters because the discriminating detail can sit anywhere in a paragraph-length option, and attention pooling beats the final hidden state for the same reason: at 250 tokens the last state is dominated by the tail.

Hyperparameters: AdamW, lr 1e-3, weight decay 1e-4, batch 32, 20 epochs, 10% linear warmup then linear decay, gradient clip 1.0, dropout 0.35, VAL_FRAC 0.15, seed 42, retriever top-k 3. A full run takes under ten minutes on one GPU.

5.2 Model 2: Fine-Tuned Transformer Model (Gemma-4 E4B IT + LoRA)

Architecture and salient points: google/gemma-4-e4b-it, an instruction-tuned decoder-only transformer loaded through Unsloth's FastLanguageModel in 4-bit NF4: RMSNorm, grouped-query attention with rotary embeddings, gated MLP blocks, tied embedding head. Instruction tuning means it already responds to "The correct answer is: "; the 4-bit path fits one Kaggle GPU; and a causal LM gives the log-probability-over-letters scoring above for free, with no generated text to parse.

Fine-tuning strategy:

Setting	Value	Setting	Value
Method	LoRA, r=8, alpha=16, dropout 0	Optimizer	adamw_8bit, lr 2e-4
Targets	v_proj, gate/up/down_proj	Schedule	cosine, 5% warmup
Base	frozen, 4-bit NF4	Precision	bf16, else fp16
Epochs	10	Memory	grad checkpointing, seq len 512
Batch	1 x 16 accum = 16 effective	Split	1600 / 400, seed 42

I targeted v_proj plus the MLP projections rather than full q/k/v/o to keep trainable parameters and runtime inside the Kaggle session limit alongside 10 epochs (~4.7 h per run).

Quantization and its consequence: My original save path merged the adapter into the 4-bit base (merge_and_unload()) and reloaded under load_in_4bit=True, scoring **0.71030**. NF4 quantizes in blocks of 64 weights normalized by their own absmax, so a LoRA delta perturbs that absmax and rescales the whole block: at my adapter's magnitude ($||\delta||/||W||$ about 0.004) the effective update had cosine similarity ~0.04 with the intended one, at ~44% of its magnitude. Model results confirmed this:

Path	MAP@3	Acc@1
4-bit base, no adapter (floor)	0.5138	0.2875
Live PEFT model, or merged in 16-bit	1.0000	1.0000
Merged then re-quantized to 4-bit (shipped)	0.5342	0.3100

The shipped path had effectively reverted the model to its base. The fix (593843f) keeps the adapter separate and applies it over the 4-bit base via PeftModel.from_pretrained, which is memory-neutral and moved the leaderboard to **0.74189**.

5.3 Model 3: Classical Baseline (TF-IDF features + logistic regression)

Deliberately simple and interpretable, to establish a floor. `build_features()` turns each (question, option) pair into 8 numbers: TF-IDF cosine similarity to the prompt, word-level Jaccard overlap, option length, length relative to the row mean, is-longest and contains-digit flags, and two within-row normalizations of the cosine. `StandardScaler` then `LogisticRegression` ($C=1$, balanced class weights) classifies each pair, and the five decision-function scores are softmaxed over A-E. Trains on CPU in about two seconds.

6. Performance & Comparative Analysis

Metrics: MAP@3 is the competition metric. I also track Accuracy@1 and macro-F1 over the five letter classes, because MAP@3 alone hides whether a model has collapsed onto the majority letters.

6.1 Training performance

W&B run happy-butterfly-23: 20 epochs, random row split

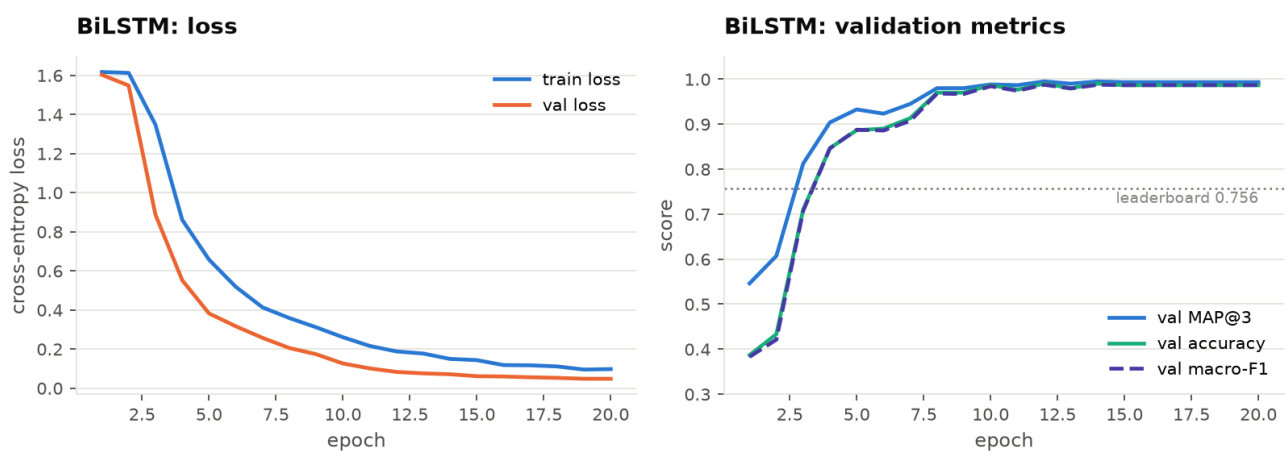


Figure 2: BiLSTM training curves

Validation loss tracks training loss the whole way down and never turns up, which would normally indicate no overfitting; Section 6.3 shows it is leakage instead. Best epoch was 12 (val MAP@3 0.9950, accuracy 0.9900, macro-F1 0.9876), and the shipped checkpoint is from that epoch. For Gemma, TRL training loss falls from ~0.60 at step 20 to ~0.0019 by epoch 10, itself a warning sign: a genuinely hard task does not reach 0.002 next-token loss.

6.2 Comparative report and Kaggle performance

	Model 1: BiLSTM (scratch)	Model 2: Gemma-4 + LoRA	Model 3: TF-IDF + LogReg
W&B run	happy-butterfly-23	driven-violet-34	model3-tfidf-logreg
Parameters	1.99 M, all trained	frozen 4-bit base + r=8 adapter	8 features, 9 coefficients
Val MAP@3	0.9950	0.9983	0.5006
Val Accuracy@1	0.9900	0.9975	0.2967
Val macro-F1	0.9876	0.9970	0.2846
Train time	~9.5 min, 1 GPU	~4.5 h, 1 GPU	~2 s, CPU
Leaderboard MAP@3	0.75644	0.74189	0.47173
Validation-to-leaderboard drop	-0.239	-0.258	-0.029

Final standing: **rank 381 of 1,531 teams**, best public MAP@3 **0.75644**. The cost/benefit is not what I expected: the BiLSTM uses about 1/30th of the LLM's training time and still finishes ahead of it, the 0.014 between them being noise. Model 3 at 0.50 validation MAP@3 is barely above the 0.4212 obtainable by always answering B C A, which is the distractor difficulty of Section 3 showing up.

The last row is the most informative. The logistic regression is the only model whose validation score survived the leaderboard, losing 0.029 where the other two lost 0.239 and 0.258. With 8 features and 9 coefficients there is nothing in it that *can* memorize an answer key, so its validation number was honest and transferred almost unchanged. It was not designed as a control, but functions as one.

6.3 Error analysis

The two deep models lose almost exactly the same amount from validation to leaderboard, despite different retrievers and separate inference code, while the logistic regression barely moves. That pattern cannot come from a model bug, so the cause is in the data.

The validation split is invalid: Both training scripts split rows at random, which across 2,000 rows covering 415 questions scatters paraphrases into both halves: **92.0%** of validation rows have a paraphrase twin in the training half. The retriever then guarantees that twin is retrieved first, via a +10.0 similarity boost on exact core-question matches, so the prompt literally contains Answer: B) <the correct text> for the same question. Since the answer letter is constant within a core group, B is also the answer for the row being scored, and the model never needs to read the options.

Where the Gemma-4 validation score actually comes from

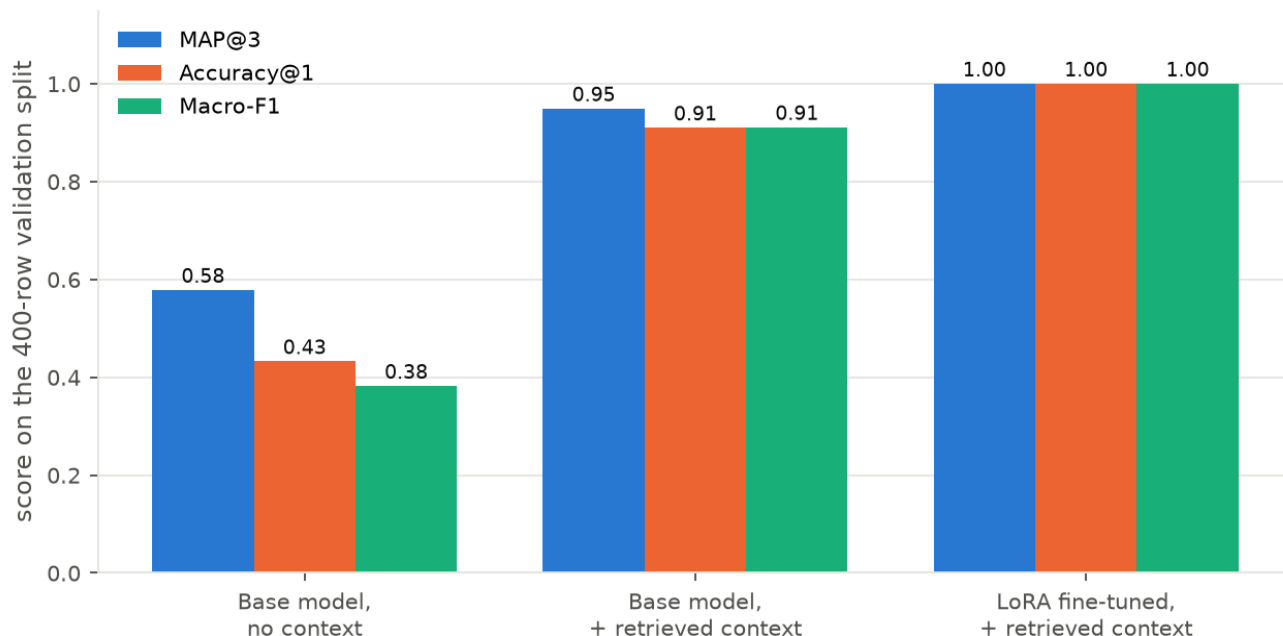


Figure 3: Gemma ablation

Configuration (same 400-row split)	MAP@3	Acc@1	Macro-F1
Base Gemma-4, no context	0.5783	0.4325	0.3821
Base Gemma-4, with retrieved context	0.9492	0.9100	0.9110
Fine-tuned, with retrieved context	1.0000	1.0000	1.0000

0.5783 is the only unaided number here: The rise to 0.9492 is retrieval supplying the answer, and the rise to 1.0000 is fine-tuning learning to copy it: ten epochs on a corpus where the answer is always visible produced a copier, not a reasoner. An earlier BiLSTM run using a *group* split on core question (distinctive-aardvark-11) peaked at 0.5700, close to Gemma's unaided 0.5783, which I take as the honest generalization estimate.

Test is more contaminated than validation, not less: My first hypothesis, that validation leaked and test did not, was wrong. Against the 1,600 rows actually trained on, test leads on every overlap measure: exact prompt 42.0% vs 16.8%, prompt-plus-options 39.8% vs 13.2%, option sequence

88.8% vs 83.8%, core question 95.0% vs 92.0%. Simulating the submission-time retriever over all 500 test rows, the retrieved answer appears at the same letter for 88.4% and at a *different* letter for **zero**, so no option reshuffling is to blame.

The models converged to a lookup table: Comparing my 0.74189 submission against {option sequence -> answer} built only from the training file: 458 of 500 test rows (91.6%) are determined by it, and on those the fine-tuned Gemma reproduces the lookup answer **456 times, 99.6%**. Two measured facts then combine. The model matches the lookup on 456 of all 500 rows and scored 0.74189, and MAP@3 is at least top-1 accuracy, so top-1 is at most 0.742; but a lookup correct on all 458 determined rows would force top-1 to at least 0.912. Therefore **the training answer key disagrees with the test key on roughly a quarter of rows carrying an identical question and an identical, identically-ordered option list**. I cannot establish why, since test labels are not released; a labelling disagreement on the ~35% of items with near-duplicate options is the most plausible explanation, though unverified.

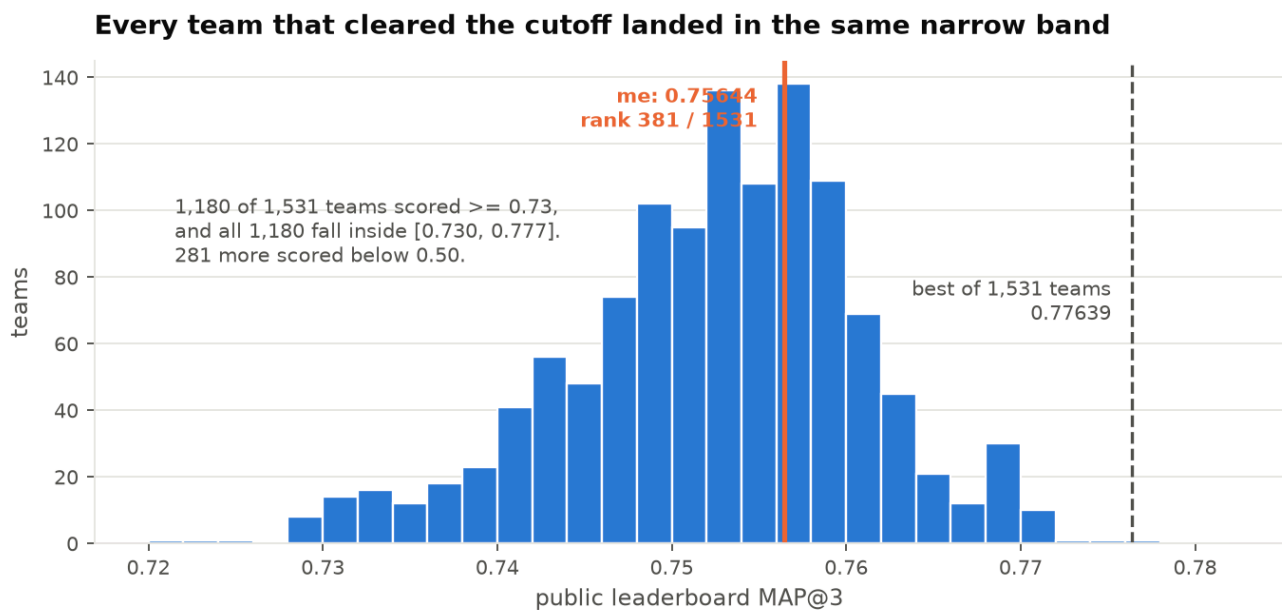


Figure 4: Leaderboard distribution

The wider leaderboard agrees: Of 1,531 teams, 1,180 cleared 0.73, and every one finished inside [0.730, 0.777]; the median is 0.75020 and the best 0.77639. Hundreds of independent attempts piled into a band 0.047 wide, which matches the internal measurement: a training-derived lookup is worth about 0.74, and no modelling reaches the rows where the two keys disagree. The gap between my score and the winner's is about ten test rows.

7. Conclusion & Future Work

Key learnings: A validation number is only as trustworthy as the split behind it. I had 0.995 and 0.998 in W&B and read them as evidence the models worked; they were measuring how completely each model had absorbed the training answer key, and a group split on core questions would have shown the real figure, near 0.57, in the first week. Capability also does not follow model size: an instruction-tuned LLM finished behind a two-million-parameter BiLSTM because neither was reasoning, both were retrieving, and fine-tuning actively removed capability by taking a base model scoring 0.5783 unaided and training that ability away. Finally, a ceiling can be a property of the task rather than the model, and the leaderboard distribution is the cheapest way to detect one.

Challenges: Kaggle session limits meant each Gemma run consumed ~4.7 h, giving roughly one experiment per session and no hyperparameter sweep, which is why the quantization issue wasn't easy. That defect was the hardest to catch: `merge_and_unload()` into a 4-bit base fails silently, and validation is computed on the live adapter before saving, so only the leaderboard reveals it.

Future work:

1. **Re-run the error analysis against the real test labels, if they are ever released.** The open question is why top-1 accuracy never reached 90%, given that the model had memorized the training set and 95% of test core questions already appeared there. Only the ground-truth key can settle whether the two answer keys genuinely disagree.
2. **Raise max_seq_length past 512**, since an estimated 15-20% of prompts exceed it and are truncated from the answer end.
3. **Try the PT checkpoint rather than the IT one.** Try the google/gemma-4-e4b (PT) checkpoint instead of the instruction-tuned one, since pretrained base models are generally recommended over IT models as a starting point for fine-tuning.

8. References

1. Hochreiter, S. & Schmidhuber, J. *Long Short-Term Memory*. Neural Computation 9(8), 1997.
2. Bahdanau, D., Cho, K. & Bengio, Y. *Neural Machine Translation by Jointly Learning to Align and Translate*. ICLR 2015. (Additive attention.)
3. Hu, E. et al. *LoRA: Low-Rank Adaptation of Large Language Models*. ICLR 2022.
4. Dettmers, T. et al. *QLoRA: Efficient Finetuning of Quantized LLMs*. NeurIPS 2023. (NF4 quantization.)
5. Lewis, P. et al. *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. NeurIPS 2020.
6. Google DeepMind. Gemma model card, google/gemma-4-e4b-it.
7. Software: PyTorch; Hugging Face transformers/datasets/peft/trl; Unsloth; bitsandbytes; scikit-learn; pandas; Weights & Biases.